

Guía de lectura activa de la Unidad 1

Consultas Oracle SQL sobre el esquema OEHR

Preparación progresiva para la primera prueba de INF-324

Guía construida desde las clases U1 L1–L6, la pauta de prueba y Oracle Database 21c

Septiembre de 2026

Nombre: _____

Fecha de prueba: Por confirmar

Objetivo de lectura

Al terminar esta guía deberías poder traducir un requerimiento escrito en español a una consulta Oracle SQL correcta, justificar cada cláusula y detectar errores frecuentes antes de comparar tu respuesta con la pauta. El foco es comprender, no memorizar consultas completas.

Cómo usar esta guía. Lee una sección corta, tapa el ejemplo, intenta reconstruirlo y recién después compáralo. Al llegar a las preguntas, escribe tus respuestas antes de mirar el solucionario. Cada módulo contiene **3 preguntas muy fáciles, 2 intermedias y 1 difícil**. Si fallas una pregunta, vuelve al párrafo que explica esa decisión, no solamente a la solución.

Conexión con la primera prueba

La evaluación histórica usada como referencia pondera un **30 %**, dispone de **90 minutos**, contiene **6 requerimientos** y usa una pauta total de **35 puntos**: 5 por formalidad y 5 por cada requerimiento. Estos datos sirven para practicar; confirma con el docente la fecha, duración y ponderación de la evaluación vigente.

Base principal del contenido local:

- scripts leccion-01.sql a leccion-06.sql;
- presentaciones leccion-01.ppt a leccion-06.ppt;
- evaluación es1-consultas-sql.docx;
- pauta revisada pauta-es1-consultas-sql.sql.

Índice

1. Mapa de la unidad y estrategia de resolución	4
1.1. Ciclo semanal para leer y practicar todos los días	4
1.2. Registro de avance	4
1.3. El método de seis decisiones	5
2. Plantillas textuales para construir consultas	5
2.1. Consulta básica y construcción de la salida	6
2.2. Filtros de filas y condiciones	6
2.3. Orden, funciones y transformación de valores	8
2.4. Resúmenes, GROUP BY y HAVING	9
2.5. Uniones entre tablas	9
2.6. Subconsultas	10
2.7. Plantilla maestra de lectura	11
3. El esquema OEHR: aprender a seguir las claves	11
3.1. Relaciones que debes reconocer	12
4. Módulo 1: SELECT, expresiones, alias y concatenación	13
4.1. SELECT y FROM	13
4.2. Expresiones aritméticas y precedencia	13
4.3. Alias de columna	14
4.4. DISTINCT	14
4.5. Concatenación	14
4.6. Lectura activa del módulo 1	14
5. Módulo 2: WHERE, operadores, patrones, nulos y orden	15
5.1. Comparaciones y texto literal	15
5.2. AND, OR, NOT y precedencia	15
5.3. BETWEEN e IN	15
5.4. LIKE y sus comodines	16
5.5. NULL no es cero ni texto vacío	16
5.6. ORDER BY	16
5.7. Lectura activa del módulo 2	17
6. Módulo 3: funciones de una sola fila	17
6.1. Funciones de caracteres	17
6.2. Funciones numéricas y DUAL	18
6.3. Aritmética y funciones de fecha	18
6.4. Lectura activa del módulo 3	19
7. Módulo 4: conversiones, formatos, nulos y decisiones	19
7.1. Conversión explícita	20
7.2. NVL y NVL2	20
7.3. COALESCE y NULLIF	20
7.4. CASE y DECODE	21
7.5. Lectura activa del módulo 4	21
8. Módulo 5: funciones de grupo, GROUP BY y HAVING	21
8.1. Funciones de grupo	22
8.2. GROUP BY	22
8.3. WHERE frente a HAVING	23
8.4. Lectura activa del módulo 5	23

9. Módulo 6: consultas de varias tablas y JOIN	24
9.1. Por qué aparece un producto cartesiano	24
9.2. INNER JOIN con ON	24
9.3. USING y NATURAL JOIN	25
9.4. LEFT OUTER JOIN	25
9.5. Self join para empleado y jefe	25
9.6. Cadena de uniones	25
9.7. Taller guiado: construir una unión desde cero	26
9.8. Unión no equitativa y CROSS JOIN	26
9.9. Lectura activa del módulo 6	26
10. Módulo 7: subconsultas	27
10.1. Subconsulta de una fila	27
10.2. Subconsulta de varias filas con IN	27
10.3. Subconsultas y valores nulos	28
10.4. Lectura activa del módulo 7	28
11. Pauta razonada de los seis requerimientos	28
11.1. Ejercicio 1: texto y patrón	28
11.2. Ejercicio 2: resumen por cargo	29
11.3. Ejercicio 3: empleado, departamento y jefe	29
11.4. Ejercicio 4: promedio del mínimo salarial por cargo	29
11.5. Ejercicio 5: departamentos de Whalen o King	29
11.6. Ejercicio 6: comisión, rango, correo y departamento de Kochhar	30
12. Lista de errores que debes poder diagnosticar	30
13. Pauta de autoevaluación alineada con la rúbrica	31
14. Simulacro acumulativo	31
15. Solucionario razonado	33
15.1. Soluciones del módulo 1	33
15.2. Soluciones del módulo 2	33
15.3. Soluciones del módulo 3	33
15.4. Soluciones del módulo 4	34
15.5. Soluciones del módulo 5	34
15.6. Soluciones del módulo 6	35
15.7. Soluciones del módulo 7	36
15.8. Soluciones del simulacro	36
16. Hoja de repaso final	37
17. Documentación oficial utilizada	38

1. Mapa de la unidad y estrategia de resolución

La Unidad 1 avanza desde una consulta de una sola tabla hasta consultas que combinan tablas, agrupan filas y emplean subconsultas. La evaluación de referencia contiene seis requerimientos y la pauta SQL revisada propone una solución para cada uno. Por eso esta guía distingue entre:

- **contenido confirmado por la pauta:** concatenación, filtros con patrones, funciones de grupo, `GROUP BY`, `HAVING`, uniones, auto-uniión y subconsultas;
- **herramientas necesarias enseñadas en clases:** consulta básica, operadores, funciones de texto, número y fecha, conversiones, nulos y ordenamiento;
- **ampliaciones presentes en las diapositivas:** `CASE`, `DECODE`, `COALESCE`, distintos tipos de `JOIN` y funciones adicionales de fecha.

1.1. Ciclo semanal para leer y practicar todos los días

Repite este ciclo hasta la fecha de la prueba. Cada sesión dura aproximadamente 40 minutos; la meta es completar al menos 5 sesiones por semana. Si un día tienes menos tiempo, haz sólo la recuperación de memoria y una consulta corta.

Día	Foco	Producto comprobable
1	Método y esquema	Dibuja las relaciones entre empleados, cargos, departamentos y jefes sin mirar la guía.
2	Módulos 1–2	Resuelve una consulta con concatenación y otra con filtros, nulos y orden.
3	Módulos 3–4	Resuelve una consulta con funciones de texto y otra con conversiones o nulos.
4	Módulo 5	Resuelve un resumen con <code>GROUP BY</code> y justifica por qué el filtro usa <code>HAVING</code> .
5	Módulo 6	Construye un <code>JOIN</code> y una auto-uniión siguiendo las claves del esquema.
6	Módulo 7	Resuelve una subconsulta de una fila y otra de varias filas.
7	Pauta completa	Simula los seis requerimientos con un límite de 90 minutos y corrige con la rúbrica.

Rutina diaria de estudio

En cada sesión usa cuatro bloques: **5 minutos** para recordar sintaxis sin mirar, **10 minutos** de lectura, **20 minutos** escribiendo SQL y **5 minutos** para corregir. Marca cada error como conceptual, de sintaxis o de lectura del enunciado. Al día siguiente comienza rehaciendo la consulta que fallaste.

1.2. Registro de avance

Marca una casilla sólo si puedes resolver el tema sin copiar el ejemplo. Usa V (verde) si lo haces solo, A (amarillo) si necesitas una pista y R (rojo) si necesitas mirar la respuesta.

Habilidad	Intento 1	Intento 2	Intento 3
SELECT , alias y concatenación WHERE , LIKE , rangos y nulos Funciones de texto, número y fecha GROUP BY y HAVING JOIN y auto-uniión Subconsultas Pauta completa en 90 minutos			

1.3. El método de seis decisiones

Antes de escribir SQL, contesta en papel:

1. **¿Qué columnas debe mostrar el resultado?** Eso construye el **SELECT**.
2. **¿De qué tablas salen?** Eso construye el **FROM**.
3. **¿Cómo se relacionan las tablas?** Eso construye cada **JOIN ... ON**.
4. **¿Qué filas individuales se aceptan?** Eso construye el **WHERE**.
5. **¿Hay que resumir filas?** Si hay conteos, promedios, mínimos, máximos o sumas, piensa en funciones de grupo y **GROUP BY**.
6. **¿Se filtran grupos o se ordena el resultado?** Usa **HAVING** para grupos y **ORDER BY** para presentación.

El orden de escritura que usaremos durante toda la guía es:

```

1 SELECT expresiones
2 FROM tablas_y_joins
3 WHERE condiciones_de_fila
4 GROUP BY columnas_de_agrupacion
5 HAVING condiciones_de_grupo
6 ORDER BY criterio_de_orden;
```

No todas las consultas necesitan todas las cláusulas, pero las que aparezcan deben conservar ese orden.

2. Plantillas textuales para construir consultas

Esta sección no entrega consultas para copiar. Cada plantilla indica, en palabras, qué debes colocar después de un comando. Los textos en cursiva representan decisiones que cambian según el enunciado; no se escriben literalmente. Lee primero el requerimiento, reemplaza mentalmente cada decisión y recién entonces redacta la consulta.

Idea clave

GROUP BY no sirve para enlazar tablas. Su función es reunir filas que comparten el mismo valor. Para enlazar una tabla con otra se utiliza **JOIN** y la relación se explica con **ON** o, cuando corresponde, con **USING**.

2.1. Consulta básica y construcción de la salida

Plantilla textual: SELECT y FROM

Qué debes colocar. Después de **SELECT**, escribe los elementos que deben aparecer como columnas del resultado. Cada elemento puede ser un atributo, un cálculo, una función o un texto fijo; separa varios elementos con comas. Después de **FROM**, escribe la tabla que contiene las filas de partida y, si será útil, asígnale un alias corto.

Frase para completar. “Mostrar *los datos solicitados* desde *la tabla que contiene las filas principales*.”

Paso a paso.

1. Subraya en el enunciado todo lo que se debe mostrar.
2. Convierte cada dato solicitado en un elemento del **SELECT**.
3. Comprueba que cada elemento esté separado por una coma.
4. Identifica la tabla que contiene las filas principales y colócala en **FROM**.
5. Si existen varias tablas, deja aquí la principal y añade las demás mediante **JOIN**.

Plantilla textual: alias con AS

Un alias cambia únicamente el encabezado mostrado o el nombre corto utilizado dentro de la consulta. Después de una columna o expresión, coloca **AS** y luego un nombre descriptivo. Si el alias contiene espacios, tildes o debe conservar exactamente mayúsculas y minúsculas, escríbelo entre comillas dobles. Para un alias de tabla se coloca el nombre corto inmediatamente después del nombre de la tabla; en Oracle no es necesario escribir **AS** para ese alias.

Plantilla textual: DISTINCT

Coloca **DISTINCT** inmediatamente después de **SELECT** cuando el resultado deba eliminar filas repetidas. La comparación considera en conjunto todos los elementos mostrados. Antes de usarlo, comprueba que la repetición no provenga de un **JOIN** incorrecto: **DISTINCT** limpia la salida, pero no repara una relación mal construida.

Plantilla textual: expresiones y concatenación

Para un cálculo, coloca en el **SELECT** el atributo numérico, el operador aritmético y el valor u otro atributo con el que se operará. Usa paréntesis cuando el orden matemático deba quedar explícito. Para formar una frase o un dato compuesto, intercala los atributos con los textos fijos y únelos con el operador de concatenación. Los textos fijos llevan comillas simples; los nombres de columnas no. Si utilizas la función **CONCAT**, recuerda que recibe dos elementos por vez, por lo que más de dos elementos requieren anidación.

2.2. Filtros de filas y condiciones

Plantilla textual: WHERE y operadores de comparación

Después de **WHERE**, coloca primero el atributo que se evaluará, luego el operador de comparación y finalmente el valor, atributo o resultado con el que se compara. Usa igualdad para una coincidencia exacta; los operadores de diferencia, mayor, menor, mayor o igual y menor o igual se eligen de acuerdo con las palabras del enunciado. Los textos y fechas convertidas explícitamente requieren el tratamiento de su tipo; los números no llevan comillas.

Frase para completar. “Conservar las filas donde *el atributo evaluado* cumpla la condición indicada.”

Paso a paso.

1. Identifica qué filas deben quedar dentro del resultado.
2. Busca el atributo que permite decidir si cada fila se acepta.
3. Traduce expresiones como “igual”, “distinto”, “superior” o “hasta” al operador correspondiente.
4. Escribe el valor de comparación con el tipo correcto.
5. Si hay más condiciones, conéctalas con **AND** u **OR** y usa paréntesis cuando pueda existir ambigüedad.

Plantilla textual: **AND**, **OR** y **NOT**

Usa **AND** cuando todas las condiciones conectadas deban cumplirse. Usa **OR** cuando sea suficiente cumplir al menos una. Coloca **NOT** delante de la condición que se desea negar. Como **AND** se evalúa antes que **OR**, agrupa con paréntesis las condiciones que expresan una misma idea. Después de escribirlas, lee la condición completa en voz alta y verifica que diga lo mismo que el requerimiento.

Plantilla textual: **BETWEEN** y **NOT BETWEEN**

Después de **BETWEEN**, coloca el límite inferior, luego **AND** y finalmente el límite superior. Ambos extremos están incluidos. Usa **NOT BETWEEN** cuando se pidan filas fuera del rango; en ese caso tampoco se incluyen los dos extremos. Verifica que ambos límites tengan un tipo compatible con el atributo evaluado.

Plantilla textual: **IN** y **NOT IN**

Después de **IN**, coloca entre paréntesis el conjunto de valores aceptados o una subconsulta que produzca esos valores. Es la opción natural cuando un atributo puede coincidir con cualquiera de varias alternativas. **NOT IN** expresa que no debe coincidir con ninguna, pero requiere cuidado si el conjunto puede contener nulos; en ese caso conviene excluirlos dentro de la subconsulta o evaluar otra estrategia.

Plantilla textual: **LIKE**

Después de **LIKE**, coloca entre comillas simples el patrón de búsqueda. El símbolo de porcentaje representa cero o más caracteres y el guion bajo representa exactamente un carácter. Coloca el porcentaje al final para “comienza con”, al inicio para “termina con” y a ambos lados para “contiene”. Si la comparación no debe depender de mayúsculas, transforma tanto el atributo como el patrón siguiendo el mismo criterio.

Plantilla textual: **IS NULL** e **IS NOT NULL**

Después del atributo coloca **IS NULL** para conservar filas donde falta el dato, o **IS NOT NULL** para conservar aquellas donde sí existe. Un nulo no es cero, una cadena vacía ni un valor que pueda compararse mediante igualdad. Primero interpreta si el enunciado pide ausencia o presencia y luego elige una de estas dos condiciones.

2.3. Orden, funciones y transformación de valores

Plantilla textual: ORDER BY

Después de **ORDER BY**, coloca el atributo, expresión o alias que determina el orden. Añade **ASC** para orden ascendente o **DESC** para descendente; si no indicas nada, se usa el orden ascendente. Para resolver empates, agrega más criterios separados por comas, desde el criterio más importante hasta el último desempate. Esta cláusula organiza la presentación y se escribe al final.

Frase para completar. “Ordenar primero por *el criterio principal* en sentido *ascendente o descendente*, y luego por *el criterio de desempate*.”

Plantilla textual: funciones de una sola fila

Coloca el nombre de la función y, entre paréntesis, el atributo o valor que debe transformar. Una función de texto recibe texto y puede cambiar mayúsculas, extraer caracteres, medir longitud o buscar una posición. Una función numérica recibe números y puede redondear, truncar u obtener un resto. Una función de fecha recibe una fecha y permite desplazarla, encontrar límites del calendario o calcular una diferencia. Estas funciones producen un resultado por cada fila y pueden anidarse: se resuelve primero la función más interna. Cuando sólo necesites evaluar una expresión y no leer datos del modelo, utiliza en **FROM** la tabla auxiliar de una fila destinada a ese propósito.

Plantilla textual: conversiones y formatos

Para convertir explícitamente, elige la función según el tipo de destino: texto, número o fecha. Dentro de sus paréntesis coloca primero el valor que se convertirá y después, cuando corresponda, el modelo de formato. Calcula y compara usando los tipos originales; aplica el formato sólo cuando el resultado esté listo para mostrarse. Esto evita ordenar números como palabras o interpretar fechas según una configuración diferente.

Plantilla textual: tratamiento de nulos

Usa **NVL** cuando necesites sustituir un posible nulo por un único valor alternativo. Usa **NVL2** cuando quieras un resultado si el dato existe y otro si falta. Usa **COALESCE** cuando debas elegir el primer valor no nulo de una lista. Usa **NULLIF** cuando dos expresiones iguales deban producir un nulo. Antes de aplicar cualquiera, decide qué significa la ausencia en el problema y comprueba que los resultados sean de tipos compatibles.

Plantilla textual: CASE y DECODE

Para una decisión general, comienza una expresión **CASE**, añade una condición con **WHEN**, coloca el resultado correspondiente después de **THEN**, repite las ramas necesarias, agrega un resultado alternativo con **ELSE** y cierra con **END**. Para comparaciones simples de igualdad, **DECODE** recibe una expresión y pares formados por valor de búsqueda y resultado, más un resultado predeterminado opcional. Prefiere **CASE** cuando existan rangos, desigualdades o varias condiciones lógicas.

2.4. Resúmenes, GROUP BY y HAVING

Plantilla textual: funciones de grupo

Coloca una función de grupo alrededor del atributo que deseas resumir. Usa **COUNT** para contar, **SUM** para sumar, **AVG** para promediar, **MIN** para obtener el menor valor y **MAX** para obtener el mayor. **COUNT(*)** cuenta filas; una función aplicada a un atributo normalmente ignora sus valores nulos. Si no existe **GROUP BY**, la función resume todo el conjunto filtrado en un único grupo.

Plantilla textual: GROUP BY

Después de **GROUP BY**, coloca el atributo o los atributos cuyos valores iguales definen cada grupo. Es decir, escribe aquello que responde “por cada qué” se desea calcular el resumen. Si el resultado muestra varios atributos normales además de funciones de grupo, todos esos atributos normales deben aparecer también en **GROUP BY**. No coloques allí las funciones de grupo.

Frase para completar. “Agrupar las filas por *el atributo que identifica cada grupo* para calcular *el conteo, suma, promedio, mínimo o máximo solicitado*.”

Paso a paso.

1. Encuentra en el enunciado expresiones como “por cada”, “por categoría” o “por tipo”.
2. Coloca ese atributo de agrupación como dato visible si el resultado debe identificar cada grupo.
3. Añade al resultado las funciones de resumen solicitadas.
4. Aplica en **WHERE** los filtros que eliminan filas antes de agrupar.
5. Después de **GROUP BY**, repite todos los atributos visibles que no estén dentro de una función de grupo.
6. Comprueba que la salida produzca una fila por cada combinación distinta de esos atributos.

Plantilla textual: HAVING

Después de **HAVING**, coloca una condición que evalúe el resultado de una función de grupo o una característica del grupo ya formado. Esta cláusula decide qué grupos permanecen. La secuencia mental es: primero filtrar filas con **WHERE**, después formar grupos con **GROUP BY**, luego calcular los resúmenes y finalmente filtrar esos grupos con **HAVING**. Si la condición puede evaluarse fila por fila antes de agrupar, normalmente corresponde a **WHERE**.

Frase para completar. “Conservar solamente los grupos cuyo *resultado agregado* cumpla la *condición solicitada*.”

2.5. Uniones entre tablas

Plantilla textual: JOIN con ON

Después de la tabla principal, coloca el tipo de **JOIN**, el nombre de la tabla secundaria y su alias. A continuación escribe **ON**; a su izquierda coloca el alias y la clave de una tabla, y a su derecha el alias y la clave relacionada de la otra. Ambas columnas deben representar la misma relación del modelo, no solamente compartir un nombre parecido.

Frase para completar. “Unir *la tabla auxiliar* con *la tabla principal* cuando *la clave de la primera* sea igual a *la clave relacionada de la segunda*.”

Paso a paso.

1. Decide cuál es la entidad principal que determina las filas que quieres listar.
2. Identifica qué dato falta y en qué tabla auxiliar se encuentra.
3. Busca en el modelo la clave que sale de la tabla principal y la clave a la que apunta.
4. Asigna un alias diferente a cada tabla.
5. Escribe en **ON** la igualdad entre ambas claves, calificadas con sus alias.
6. Repite el proceso por cada tabla adicional y revisa si alguna unión multiplica filas de forma inesperada.

Plantilla textual: INNER, LEFT, RIGHT y FULL JOIN

Usa **INNER JOIN**, o simplemente **JOIN**, cuando sólo deban permanecer filas con coincidencia en ambas tablas. Usa **LEFT JOIN** cuando deban conservarse todas las filas de la tabla escrita a la izquierda, aunque no tengan coincidencia. **RIGHT JOIN** conserva todas las filas de la tabla derecha. **FULL JOIN** conserva las filas de ambos lados. Primero identifica qué conjunto debe preservarse y luego ubícalo en el lado coherente con el tipo de unión elegido.

Plantilla textual: USING, NATURAL JOIN, auto-unión y cadena de uniones

Usa **USING** únicamente cuando las dos tablas comparten una columna de enlace con exactamente el mismo nombre; dentro de sus paréntesis coloca ese nombre una sola vez. Para una auto-unión, escribe la misma tabla dos veces con alias distintos, asignando a cada copia un rol diferente, y relaciona esos roles mediante **ON**. Para una cadena, añade una tabla a la vez siguiendo el camino real de claves y verifica cada enlace antes de continuar con el siguiente. **NATURAL JOIN** busca de forma automática todas las columnas con nombres coincidentes; úsalo sólo si el modelo confirma que todas esas coincidencias representan las relaciones deseadas, porque puede enlazar columnas adicionales sin que resulte evidente.

Plantilla textual: unión no equitativa y CROSS JOIN

Una unión no equitativa utiliza en **ON** una condición de rango o desigualdad en lugar de una igualdad, cuando el modelo define la coincidencia de esa forma. **CROSS JOIN** combina cada fila de una tabla con todas las filas de la otra y no utiliza condición de enlace. Debe aparecer sólo cuando el producto cartesiano sea solicitado expresamente; si surge por accidente, revisa las claves de unión.

2.6. Subconsultas**Plantilla textual: subconsulta de una fila**

Dentro de paréntesis, coloca una consulta interna que responda primero la pregunta auxiliar y devuelva un único valor. En la consulta externa, compara un atributo con ese resultado mediante el operador simple apropiado. Antes de usar igualdad, mayor o menor, asegúrate de que la consulta interna no pueda devolver más de una fila; de lo contrario se producirá un error.

Frase para completar. “Conservar las filas cuyo *atributo externo* se compare con *el único valor calculado por la pregunta interna*.”

Plantilla textual: subconsulta de varias filas

Dentro de paréntesis, coloca una consulta interna que produzca una columna con cero, uno o varios valores. En la consulta externa, utiliza un operador preparado para conjuntos, como IN, para comparar el atributo con cualquiera de esos resultados. La consulta interna debe mostrar sólo la columna necesaria para la comparación y sus valores deben ser compatibles con el atributo externo.

Frase para completar. “Conservar las filas cuyo *atributo externo* pertenezca al *conjunto de valores obtenido por la pregunta interna*.”

Plantilla textual: método para construir una subconsulta

1. Separa el requerimiento en una pregunta principal y una pregunta auxiliar.
2. Resuelve primero la pregunta auxiliar y determina cuántas filas podría devolver.
3. Coloca esa consulta entre paréntesis en la condición de la pregunta principal.
4. Usa un operador simple para un solo valor o un operador de conjunto para varios valores.
5. Controla los nulos cuando puedan alterar una comparación negativa.
6. Ejecuta mentalmente primero la parte interna y después la externa para justificar el resultado.

2.7. Plantilla maestra de lectura

Para cualquier requerimiento, completa en una hoja las siguientes frases antes de escribir SQL:

1. Mostraré *los atributos, cálculos o textos pedidos*.
2. La fila principal proviene de *la tabla que representa lo que se desea listar*.
3. Añadiré *cada tabla auxiliar* mediante *su clave de enlace real*.
4. Conservaré únicamente las filas que cumplan *las condiciones individuales*.
5. Resumiré con *la función solicitada* por cada *atributo de agrupación*, si corresponde.
6. Conservaré los grupos que cumplan *la condición agregada*.
7. Presentaré el resultado según *el criterio y sentido de orden solicitado*.

Cuando una frase no aplica, omite su cláusula. Las frases que sí aplican deben transformarse respetando el orden de escritura de la sentencia.

3. El esquema OEHR: aprender a seguir las claves

Una consulta de varias tablas sólo es correcta si sus relaciones representan el modelo. Las tablas más importantes para la prueba son las siguientes.

Cómo leer un esquema antes de consultar

Una base de datos relacional separa la información en tablas para no repetir innecesariamente los mismos datos. En OEHR, por ejemplo, la fila de un empleado no guarda el texto completo del departamento ni el nombre de su cargo: guarda DEPARTMENT_ID y JOB_ID. Esos valores funcionan como referencias hacia las filas correspondientes de OEHR_DEPARTMENTS y OEHR_JOBS. Para resolver

un requerimiento debes distinguir la *tabla principal*, que representa aquello que el enunciado quiere listar, de las tablas auxiliares que aportan atributos. Si el enunciado dice “muestre todos los trabajadores con su departamento”, la tabla principal es `OEHR_EMPLOYEES`: quieres obtener una fila de salida por trabajador. Como el nombre del departamento no está allí, sigues la referencia `EMPLOYEES.DEPARTMENT_ID` hasta la clave `DEPARTMENTS.DEPARTMENT_ID`. La igualdad no se inventa por semejanza semántica, sino que se deduce de la clave foránea declarada en el esquema. En esta relación, muchos empleados pueden pertenecer a un mismo departamento, pero cada empleado apunta como máximo a un departamento. Por eso, al unir correctamente ambas tablas, un empleado no debería multiplicarse por todos los departamentos. Este razonamiento también permite anticipar nulos: `DEPARTMENT_ID` admite nulo, por lo que podría existir un empleado sin departamento asignado. Si el requerimiento exige conservarlo, la unión tendrá que preservar la tabla principal mediante `LEFT JOIN`; si sólo interesan las coincidencias, bastará un `JOIN` interno. Cuando la información solicitada está más lejos, no se salta directamente entre tablas: se recorre el camino completo de claves. Para llegar desde empleado a región se pasa por departamento, ubicación y país. Pensar el esquema como un mapa de nodos y enlaces evita dos fallos comunes: unir columnas que no representan la misma entidad y ocultar un error con `DISTINCT`. `DISTINCT` puede quitar duplicados visibles, pero jamás corrige una relación mal construida.

Tabla	Columnas relevantes
<code>OEHR_EMPLOYEES</code>	<code>EMPLOYEE_ID</code> , <code>FIRST_NAME</code> , <code>LAST_NAME</code> , <code>EMAIL</code> , <code>HIRE_DATE</code> , <code>JOB_ID</code> , <code>SALARY</code> , <code>COMMISSION_PCT</code> , <code>MANAGER_ID</code> , <code>DEPARTMENT_ID</code> .
<code>OEHR_JOBS</code>	<code>JOB_ID</code> , <code>JOB_TITLE</code> , <code>MIN_SALARY</code> , <code>MAX_SALARY</code> .
<code>OEHR_DEPARTMENTS</code>	<code>DEPARTMENT_ID</code> , <code>DEPARTMENT_NAME</code> , <code>MANAGER_ID</code> , <code>LOCATION_ID</code> .
<code>OEHR_LOCATIONS</code>	<code>LOCATION_ID</code> , <code>STREET_ADDRESS</code> , <code>POSTAL_CODE</code> , <code>CITY</code> , <code>STATE_PROVINCE</code> , <code>COUNTRY_ID</code> .
<code>OEHR_COUNTRIES</code>	<code>COUNTRY_ID</code> , <code>COUNTRY_NAME</code> , <code>REGION_ID</code> .
<code>OEHR_REGIONS</code>	<code>REGION_ID</code> , <code>REGION_NAME</code> .

3.1. Relaciones que debes reconocer

```

EMPLOYEES.JOB_ID = JOBS.JOB_ID
EMPLOYEES.DEPARTMENT_ID = DEPARTMENTS.DEPARTMENT_ID
EMPLOYEES.MANAGER_ID = EMPLOYEES.EMPLOYEE_ID
DEPARTMENTS.LOCATION_ID = LOCATIONS.LOCATION_ID
LOCATIONS.COUNTRY_ID = COUNTRIES.COUNTRY_ID
COUNTRIES.REGION_ID = REGIONS.REGION_ID

```

La tercera relación es especial: una fila de empleados apunta a otra fila de la misma tabla. Eso permite encontrar al jefe mediante una *auto-uniión* o *self join*.

Pausa activa

Sin escribir SQL, traza el camino necesario para mostrar el apellido de un empleado y el continente donde trabaja. La ruta es: empleado → departamento → ubicación → país → región.

4. Módulo 1: SELECT, expresiones, alias y concatenación

Objetivo de lectura

Construir la salida de una consulta de una tabla, calcular columnas y producir textos legibles.

El primer paso de toda consulta consiste en definir con exactitud la forma del resultado. Una sentencia **SELECT** no modifica los datos: construye una tabla temporal de salida a partir de las filas disponibles en **FROM**. Cada elemento separado por coma dentro de **SELECT** se convierte en una columna del resultado y puede ser una columna almacenada, un texto fijo, una operación aritmética o una función. Esta idea ayuda a traducir enunciados: si se solicita “código, nombre completo y sueldo anual”, aparecen tres expresiones, aunque el nombre completo use dos columnas y el sueldo anual se calcule multiplicando por doce. El resultado no necesita copiar la estructura física de la tabla. Puede presentar datos derivados y encabezados claros mediante alias. Sin embargo, la consulta se evalúa para cada fila producida por **FROM**; por eso una expresión como **salary*12** genera un valor anual para cada empleado, no un total de la empresa. Para obtener totales se requieren funciones de grupo, estudiadas más adelante. También debes diferenciar una constante de una columna: **'Empleado:'** es texto literal porque está entre comillas simples, mientras que **last_name** es una referencia al dato de la fila actual. La concatenación combina esas piezas para construir reportes humanos sin alterar los valores originales. Los paréntesis son decisivos en cálculos: **(salary+100)*12** aplica un bono mensual antes de anualizar, mientras que **salary+100*12** sólo suma 1200 al sueldo mensual por la precedencia de la multiplicación. Finalmente, **DISTINCT** debe entenderse sobre la fila de salida completa. Si seleccionas únicamente **job_id**, obtienes cargos distintos; si seleccionas además **employee_id**, cada empleado vuelve a diferenciar la fila. Antes de ejecutar, cuenta las expresiones solicitadas, verifica las comas y pregúntate qué valor produciría cada una para un solo empleado concreto. Esa simulación mental detecta muchos errores tempranos.

4.1. SELECT y FROM

SELECT indica qué se mostrará y **FROM** de dónde se obtendrán las filas.

```
1 SELECT employee_id, first_name, salary
2 FROM oehr_employees;
```

El asterisco devuelve todas las columnas. Es útil para explorar, pero en una respuesta de prueba es mejor nombrar únicamente las columnas solicitadas.

```
1 SELECT *
2 FROM oehr_departments;
```

4.2. Expresiones aritméticas y precedencia

Las columnas numéricas pueden participar en expresiones con **+**, **-**, ***** y **/**. Multiplicación y división se evalúan antes que suma y resta; usa paréntesis para expresar la intención.

```
1 SELECT employee_id,
2         salary,
3         salary * 12 AS sueldo_anual,
4         (salary + 57) * 12 AS anual_con_bono_mensual
5 FROM oehr_employees;
```

Un nulo propagará normalmente el nulo en una expresión. Por ejemplo, **salary * commission_pct** será nulo cuando la comisión sea nula. Más adelante se resolverá con **NVL**.

4.3. Alias de columna

Un alias cambia el encabezado del resultado, no el nombre almacenado de la columna. Puede escribirse con AS. Las comillas dobles conservan espacios, tildes o mayúsculas exactas.

```
1 SELECT employee_id AS "COD_EMPLEADO",
2       salary * 12 AS "SUELDO_ANUAL"
3 FROM oehr_employees;
```

No uses comillas simples para un alias: en Oracle las comillas simples representan texto literal.

4.4. DISTINCT

DISTINCT elimina filas duplicadas de la combinación completa de expresiones seleccionadas.

```
1 SELECT DISTINCT job_id
2 FROM oehr_employees;
```

4.5. Concatenación

Oracle permite concatenar con ||. También ofrece CONCAT, pero ésta recibe dos argumentos; para unir tres piezas debe anidarse.

```
1 SELECT employee_id,
2       first_name || ' ' || last_name AS nombre_completo
3 FROM oehr_employees;
4
5 SELECT employee_id,
6       CONCAT(first_name, CONCAT(' ', last_name)) AS nombre_completo
7 FROM oehr_employees;
```

Conexión con la primera prueba

La primera consulta de la pauta pide formar una frase con JOB_TITLE y JOB_ID. La construcción más clara es:

```
1 SELECT 'El codigo del cargo ' || job_title ||
2       ' es ' || job_id AS reporte
3 FROM oehr_jobs;
```

Después se añadirá el filtro que exige que el título o el código comience con S.

4.6. Lectura activa del módulo 1

M1.1 Muy fácil. ¿Qué cláusula indica las columnas que se mostrarán?

M1.2 Muy fácil. ¿Qué símbolo permite pedir todas las columnas?

M1.3 Muy fácil. ¿Qué palabra elimina resultados duplicados?

M1.4 Intermedia. Escribe una consulta que muestre código, apellido y sueldo anual de cada empleado, con los alias CODIGO, APELLIDO y ANUAL.

M1.5 Intermedia. Explica la diferencia entre $\text{salary} + 100 * 12$ y $(\text{salary} + 100) * 12$.

M1.6 Difícil. Genera una sola columna con el texto “KING gana 24000 al año”, usando el apellido real, el sueldo real y una expresión anual. No escribas valores fijos salvo los textos de unión.

5. Módulo 2: WHERE, operadores, patrones, nulos y orden

Objetivo de lectura

Seleccionar filas individuales con precisión y presentar el resultado en el orden solicitado.

Una consulta suele comenzar con muchas filas posibles y luego descartar las que no cumplen el requerimiento. WHERE expresa precisamente esa decisión fila por fila: para cada empleado, Oracle evalúa una condición cuyo resultado lógico puede ser verdadero, falso o desconocido. Sólo las filas verdaderas continúan. La tercera posibilidad aparece principalmente con nulos y explica por qué `commission_pct = NULL` no funciona: un valor desconocido no puede afirmarse igual a otro valor desconocido; debe preguntarse `IS NULL`. Al combinar condiciones conviene traducir el español de manera literal. “Empleados del departamento 50 u 80, y con sueldo mayor a 5000” se convierte en `department_id IN (50,80) AND salary>5000`. Los paréntesis protegen la intención cuando existe una mezcla de AND y OR; sin ellos, la precedencia puede aceptar filas que el enunciado no quería. Los operadores abreviados también tienen significado exacto: BETWEEN incluye los dos límites e IN representa una lista de alternativas. En texto, LIKE compara un patrón, no una frase aproximada. El porcentaje acepta cualquier cantidad de caracteres y el guion bajo exactamente uno, de modo que `'_a%s'` exige una `a` en la segunda posición y una `s` al final. Para no depender de cómo estén escritas las mayúsculas, transforma tanto el razonamiento como el patrón a un mismo caso, por ejemplo `UPPER(last_name) LIKE 'S%'`. Una vez elegidas las filas, ORDER BY modifica solamente su presentación: no cambia qué filas existen ni arregla un filtro equivocado. Cuando se ordena por varias columnas, la primera define los grupos principales de orden y las siguientes resuelven los empates. Leer una consulta correctamente significa imaginar esta secuencia: obtener filas, evaluar cada condición, conservar las verdaderas y recién entonces ordenar lo que quedó.

5.1. Comparaciones y texto literal

WHERE conserva las filas cuya condición resulta verdadera. Los textos y fechas escritas como literales se rodean con comillas simples; los números no.

```
1 SELECT employee_id, last_name, salary
2 FROM oehr_employees
3 WHERE salary >= 10000;
4
5 SELECT employee_id, last_name
6 FROM oehr_employees
7 WHERE last_name = 'Whalen';
```

Operadores comunes: =, <>, >, <, >= y <=.

5.2. AND, OR, NOT y precedencia

AND exige que ambas condiciones sean verdaderas; OR exige al menos una; NOT niega. Oracle evalúa NOT, luego AND y luego OR. Usa paréntesis cuando el requerimiento mezcle operadores.

```
1 SELECT employee_id, last_name, job_id, salary
2 FROM oehr_employees
3 WHERE (job_id = 'AD_PRES' OR job_id = 'SA_REP')
4 AND salary > 15000;
```

5.3. BETWEEN e IN

BETWEEN incluye ambos extremos. IN compara con un conjunto de alternativas.

```
1 SELECT employee_id, last_name, salary
2 FROM oehr_employees
```

```

3 WHERE salary BETWEEN 3000 AND 5000;
4
5 SELECT employee_id, last_name, department_id
6 FROM oehr_employees
7 WHERE department_id IN (50, 80, 90);

```

Para excluir un conjunto usa NOT IN, no una cadena de desigualdades unida con OR. La expresión `x <>100 OR x <>120` es prácticamente siempre verdadera.

5.4. LIKE y sus comodines

LIKE compara patrones de texto:

- % representa cero o más caracteres;
- _ representa exactamente un carácter.

```

1 -- Apellido que comienza con S
2 WHERE last_name LIKE 'S%'
3
4 -- Nombre de departamento cuya segunda letra es a y termina en s
5 WHERE LOWER(department_name) LIKE '_a%s'
6
7 -- Cargo cuya penultima letra es a
8 WHERE LOWER(job_id) LIKE '%a_'

```

Las comparaciones de texto distinguen mayúsculas y minúsculas según los valores y la configuración. Una técnica del curso es normalizar el dato con UPPER o LOWER y comparar con un patrón en el mismo caso.

```

1 WHERE UPPER(job_title) LIKE 'S%'
2    OR UPPER(job_id) LIKE 'S%'

```

5.5. NULL no es cero ni texto vacío

Un nulo representa un valor desconocido, ausente o no aplicable. No se evalúa con `= NULL` ni `<>NULL`.

```

1 SELECT employee_id, last_name
2 FROM oehr_employees
3 WHERE commission_pct IS NULL;

```

Usa IS NOT NULL para exigir un valor conocido.

5.6. ORDER BY

ORDER BY se escribe al final. El orden predeterminado es ascendente; DESC lo invierte. Se puede ordenar por columna, alias o posición, aunque el nombre o alias suele ser más legible.

```

1 SELECT employee_id, last_name, department_id, hire_date
2 FROM oehr_employees
3 WHERE department_id IN (50, 80)
4 ORDER BY department_id ASC, hire_date DESC, last_name ASC;

```

Error frecuente

No coloques punto y coma antes de ORDER BY. El punto y coma termina la sentencia completa.

5.7. Lectura activa del módulo 2

M2.1 Muy fácil. ¿Qué operador comprueba si una columna es nula?

M2.2 Muy fácil. En LIKE, ¿qué representa `_`?

M2.3 Muy fácil. ¿Qué palabra ordena de mayor a menor?

M2.4 Intermedia. Muestra empleados de los departamentos 50 u 80 cuyo sueldo esté entre 4000 y 9000, incluidos los límites.

M2.5 Intermedia. Escribe el patrón para apellidos cuya segunda letra sea `a` y cuya última letra sea `s`, ignorando mayúsculas.

M2.6 Difícil. Muestra empleados que pertenezcan a los departamentos 50, 80 o 90 y que además no tengan comisión, ordenados por departamento ascendente y sueldo descendente.

6. Módulo 3: funciones de una sola fila

Objetivo de lectura

Transformar texto, números y fechas sin cambiar la cantidad de filas de entrada.

Las funciones de una sola fila son pequeñas transformaciones aplicadas independientemente a cada registro. Si entran 107 empleados y no existe un filtro, una expresión con `UPPER`, `SUBSTR` o `ROUND` sigue produciendo 107 resultados; no resume el conjunto. Para decidir qué función usar, identifica primero el tipo de dato que tienes y el tipo de resultado que necesitas. Con texto puedes cambiar capitalización, medir longitud, buscar una posición o extraer un fragmento. Por ejemplo, crear un correo exige descomponer el requerimiento en piezas: extraer la primera letra con `SUBSTR(first_name,1,1)`, añadir el apellido y finalmente el dominio. Con números, `ROUND` y `TRUNC` responden a preguntas diferentes: el primero aproxima al valor más cercano y el segundo elimina las posiciones que sobran. Con fechas, Oracle almacena un valor de fecha, no simplemente el texto que ves en pantalla. Por ello puede restar fechas para obtener días, sumar días o emplear funciones que navegan por meses. Si calculas semanas trabajadas, la resta `SYSDATE-hire_date` entrega días, la división por siete los transforma en semanas y `TRUNC` elimina la parte de semana incompleta. Este razonamiento por unidades evita usar funciones al azar. `DUAL` aparece cuando deseas evaluar una expresión sin recorrer una tabla de negocio: permite comprobar cuánto devuelve `ROUND(45.5674,2)` o cuál es la fecha actual. Las funciones pueden anidarse, y Oracle evalúa desde la más interna hacia afuera. En `UPPER(SUBSTR(first_name,1,1))`, primero se extrae y luego se convierte. Antes de escribir una expresión compleja, dibuja sus pasos y comprueba el tipo que sale de cada uno. Una función correcta con argumentos incorrectos puede producir un resultado válido sintácticamente, pero distinto del que el problema solicita.

Una función de una sola fila recibe valores de una fila y devuelve un resultado por esa fila. Puede usarse en `SELECT`, `WHERE` y `ORDER BY`, respetando los tipos de datos.

6.1. Funciones de caracteres

Función	Idea y ejemplo
<code>UPPER(texto)</code>	Convierte a mayúsculas: <code>UPPER(last_name)</code> .
<code>LOWER(texto)</code>	Convierte a minúsculas: <code>LOWER(email)</code> .
<code>INITCAP(texto)</code>	Coloca en mayúscula la inicial de cada palabra.
<code>CONCAT(a,b)</code>	Une dos valores; para más valores, anida o usa <code> </code> .
<code>SUBSTR(t,p,n)</code>	Extrae <code>n</code> caracteres de <code>t</code> desde la posición <code>p</code> . Una posición negativa cuenta desde el final.

Función	Idea y ejemplo
LENGTH(texto)	Devuelve la cantidad de caracteres.
INSTR(t,b)	Devuelve la posición donde comienza b dentro de t; si no lo encuentra, devuelve cero.

Ejemplo del estilo de la clase: crear un correo con la primera letra del nombre, el apellido y un dominio.

```

1 SELECT employee_id,
2         UPPER(SUBSTR(first_name, 1, 1) ||
3                last_name || '@SQL.CL') AS email_propuesto
4 FROM oeher_employees;
```

Para extraer el último carácter se puede usar SUBSTR(texto,-1,1). Para quitar el penúltimo carácter conviene razonar en segmentos: parte anterior, último carácter y dominio.

6.2. Funciones numéricas y DUAL

ROUND redondea, TRUNC corta sin redondear y MOD calcula el resto de una división.

```

1 SELECT ROUND(45.5674, 2) AS redondeado,
2         TRUNC(45.5674, 2) AS truncado,
3         MOD(54, 5) AS resto
4 FROM dual;
```

Resultado conceptual: 45.57, 45.56 y 4. DUAL es una tabla especial de una fila útil cuando la expresión no depende de datos de una tabla del usuario.

Error frecuente

La sintaxis correcta es MOD(54,3), no MOD(54/3). En una sesión configurada con punto decimal, escribe 45.5674; una coma separaría argumentos.

6.3. Aritmética y funciones de fecha

En Oracle, sumar un número a una fecha agrega días y restar dos fechas produce una diferencia en días. SYSDATE devuelve la fecha y hora actuales del servidor.

```

1 SELECT employee_id,
2         hire_date,
3         TRUNC((SYSDATE - hire_date) / 7) AS semanas_trabajadas
4 FROM oeher_employees
5 WHERE department_id IN (50, 80)
6 ORDER BY semanas_trabajadas;
```

Otras funciones del material:

- MONTHS_BETWEEN(fecha1, fecha2): diferencia aproximada en meses;
- ADD_MONTHS(fecha, n): suma o resta meses;
- NEXT_DAY(fecha, 'MONDAY'): siguiente día indicado posterior a la fecha;
- LAST_DAY(fecha): último día del mes;
- ROUND(fecha, formato) y TRUNC(fecha, formato): redondean o truncan fechas.

```

1 SELECT employee_id,
2        hire_date,
3        ADD_MONTHS(hire_date, 6) AS revision,
4        LAST_DAY(hire_date) AS fin_mes_contrato
5 FROM oeohr_employees;

```

Los nombres de días empleados por `NEXT_DAY` pueden depender del idioma de la sesión. En una prueba, sigue la configuración indicada por el profesor.

6.4. Lectura activa del módulo 3

M3.1 Muy fácil. ¿Qué función convierte un texto a mayúsculas?

M3.2 Muy fácil. ¿Qué diferencia esencial existe entre `ROUND` y `TRUNC`?

M3.3 Muy fácil. ¿Para qué se usa `DUAL`?

M3.4 Intermedia. Construye un correo formado por las dos primeras letras del nombre, un punto, el apellido y `@ucm.cl`, todo en minúsculas.

M3.5 Intermedia. Calcula los años completos aproximados trabajados dividiendo los meses obtenidos con `MONTHS_BETWEEN` por 12 y truncando el resultado.

M3.6 Difícil. Genera el identificador formado por la última letra del apellido, la primera letra del nombre y la longitud total de ambos nombres concatenados, todo en mayúsculas.

7. Módulo 4: conversiones, formatos, nulos y decisiones

Objetivo de lectura

Controlar explícitamente los tipos de datos, mostrar fechas y dinero con formato y producir valores alternativos cuando existan nulos.

Los datos poseen tipos y Oracle necesita saber qué operaciones tienen sentido para cada tipo. Un número puede sumarse, una fecha puede compararse cronológicamente y un texto puede concatenarse. A veces Oracle intenta convertir automáticamente un tipo en otro, pero esa conversión implícita depende de la configuración de la sesión. El texto `'01/02/2025'` podría interpretarse como primero de febrero o dos de enero; por eso una consulta estable usa `TO_DATE` con un formato explícito. `TO_CHAR` realiza el camino contrario cuando el objetivo es presentar el valor, no seguir calculando con él. Formatear `salary` como dinero produce texto visual; si después necesitas promediar, debes conservar el número original para el cálculo y formatear sólo el resultado final. Los nulos exigen otra decisión. `NVL(commission_pct,0)` afirma una regla de negocio para esa expresión: cuando la comisión falta, será tratada como cero. Eso permite multiplicar el sueldo sin que el nulo se propague. No significa que la base de datos haya guardado cero ni que ambos conceptos sean siempre equivalentes. `NVL2` y `CASE` permiten producir mensajes diferentes según la existencia o el valor de un dato. En una rama condicional, todos los resultados deben poder convertirse a tipos compatibles. Si una rama devuelve dinero formateado y otra el texto `'SIN COMISION'`, conviene que ambas devuelvan texto: se aplica `TO_CHAR` al monto en la primera rama. La regla práctica es separar tres fases: calcular usando números o fechas reales, decidir usando condiciones y formatear al final para mostrar. Esta separación evita comparar fechas como palabras, ordenar montos por su símbolo monetario o sumar cadenas que sólo parecen números.

7.1. Conversión explícita

Oracle puede hacer algunas conversiones implícitas, pero dependen de la configuración y pueden fallar. Una solución robusta expresa la conversión.

Función	Uso
TO_CHAR(fecha,formato)	Convierte una fecha en texto controlando su representación.
TO_CHAR(número,formato)	Convierte un número en texto, por ejemplo para moneda.
TO_NUMBER(texto,formato)	Convierte texto numérico en número.
TO_DATE(texto,formato)	Interpreta texto como fecha usando un modelo explícito.

```

1 SELECT employee_id,
2         TO_CHAR(hire_date, 'DD "DE" MONTH "DE" YYYY') AS fecha,
3         TO_CHAR(salary, 'FM$999,999,990.00') AS sueldo
4 FROM oehr_employees;
```

FM reduce los espacios de relleno del formato. Las palabras literales dentro de un formato de fecha se rodean con comillas dobles dentro del texto SQL.

```

1 SELECT employee_id, last_name, hire_date
2 FROM oehr_employees
3 WHERE hire_date >= TO_DATE('2016-01-01', 'YYYY-MM-DD')
4 ORDER BY hire_date;
```

Esto es preferible a comparar con '01/01/2016', porque el significado de ese texto depende de NLS_DATE_FORMAT.

7.2. NVL y NVL2

NVL(valor,reemplazo) devuelve el reemplazo sólo si el valor es nulo. Ambos deben ser de tipos compatibles.

```

1 SELECT employee_id,
2         salary,
3         NVL(commission_pct, 0) AS comision,
4         salary * NVL(commission_pct, 0) AS monto_comision
5 FROM oehr_employees;
```

NVL2(valor,si_no_es_nulo,si_es_nulo) elige entre dos resultados.

```

1 SELECT employee_id,
2         NVL2(commission_pct,
3             TO_CHAR(commission_pct * 100, 'FM990') || '%',
4             'SIN COMISION') AS estado_comision
5 FROM oehr_employees;
```

7.3. COALESCE y NULLIF

COALESCE(a,b,c) devuelve el primer valor no nulo. NULLIF(a,b) devuelve nulo cuando ambos valores son iguales; en caso contrario devuelve a.

```

1 SELECT employee_id,
2         COALESCE(TO_CHAR(commission_pct),
3                 TO_CHAR(manager_id),
4                 'SIN COMISION NI JEFE') AS dato_disponible
5 FROM oehr_employees;
```

7.4. CASE y DECODE

CASE permite expresar decisiones legibles y es parte del estándar SQL.

```

1 SELECT employee_id, salary,
2     CASE
3         WHEN salary >= 15000 THEN 'ALTO'
4         WHEN salary >= 8000 THEN 'MEDIO'
5         ELSE 'BAJO'
6     END AS tramo_sueldo
7 FROM oeher_employees;
```

DECODE es una función propia de Oracle útil para equivalencias simples.

```

1 SELECT employee_id, department_id,
2     DECODE(department_id,
3         10, 'ADMINISTRACION',
4         20, 'MARKETING',
5         'OTRO') AS area
6 FROM oeher_employees;
```

Para rangos y condiciones compuestas suele ser más claro CASE. Para una equivalencia corta, DECODE puede ser suficiente.

7.5. Lectura activa del módulo 4

M4.1 Muy fácil. ¿Qué función convierte una fecha en texto formateado?

M4.2 Muy fácil. ¿Qué devuelve NVL(NULL,0)?

M4.3 Muy fácil. ¿Qué función devuelve el primer argumento no nulo de una lista?

M4.4 Intermedia. Muestra el sueldo con dos decimales y la fecha de contratación con el formato YYYY-MM-DD.

M4.5 Intermedia. Muestra CON COMISION o SIN COMISION según commission_pct, utilizando CASE.

M4.6 Difícil. Muestra nombre completo, sueldo con formato monetario y monto de comisión con formato monetario; quienes no tengan comisión deben mostrar SIN COMISION en la última columna. Evita mezclar números y textos incompatibles.

8. Módulo 5: funciones de grupo, GROUP BY y HAVING

Objetivo de lectura

Resumir muchas filas en resultados por empresa, departamento o cargo, y filtrar esos resultados.

Agrupar significa cambiar la unidad de análisis. En una consulta común, la unidad es una fila de empleado; después de GROUP BY department_id, la unidad pasa a ser un departamento compuesto por todas sus filas de empleados. Las funciones de grupo responden preguntas sobre ese conjunto: cuántos miembros tiene, cuánto suman sus sueldos, cuál es el promedio o cuáles son los extremos. Esta transformación explica la regla de GROUP BY: si el resultado representa una fila por departamento, Oracle no puede mostrar libremente last_name, porque dentro del grupo existen muchos apellidos y no hay una única respuesta. Debes agrupar también por apellido, resumirlo con una función apropiada o retirarlo. El orden conceptual ayuda a ubicar filtros. WHERE actúa cuando todavía existen empleados individuales; por eso sirve para limitar el estudio a departamentos 50, 80 y 90 antes de agrupar. HAVING actúa cuando ya existe cada grupo y sus valores calculados; por eso puede preguntar

si `AVG(salary)>8000` o si `COUNT(*)>=2`. En la pauta, la diferencia `MAX(salary)-MIN(salary)` sólo existe después de reunir los empleados de cada cargo, de modo que debe escribirse en `HAVING`. También hay que pensar cuidadosamente en nulos: `COUNT(*)` cuenta filas, mientras `COUNT(commission_pct)` cuenta únicamente valores de comisión conocidos. Del mismo modo, `AVG(commission_pct)` ignora empleados sin comisión, pero `AVG(NVL(commission_pct,0))` los incorpora como cero y responde otra pregunta. Para verificar una consulta agrupada, toma un grupo imaginario de tres filas, calcula manualmente la función y pregunta si cada columna visible posee exactamente un valor para ese grupo. Si no lo posee, el diseño del `SELECT` y del `GROUP BY` aún no está completo.

8.1. Funciones de grupo

- `COUNT(*)`: cuenta filas.
- `COUNT(columna)`: cuenta valores no nulos de esa columna.
- `COUNT(DISTINCT columna)`: cuenta valores diferentes no nulos.
- `SUM(columna)`: suma valores no nulos.
- `AVG(columna)`: promedio de valores no nulos.
- `MIN(columna)` y `MAX(columna)`: extremos del conjunto.

```

1 SELECT ROUND(AVG(salary), 0) AS promedio,
2         MIN(hire_date) AS primera_contratacion,
3         MAX(hire_date) AS ultima_contratacion,
4         COUNT(*) AS cantidad_filas
5 FROM oehr_employees;
```

Las funciones ignoran nulos salvo `COUNT(*)`, que cuenta filas. Esto importa al promediar comisiones. `AVG(commission_pct)` promedia únicamente empleados con comisión; `AVG(NVL(commission_pct,0))` incluye al resto como cero. No significan lo mismo.

8.2. GROUP BY

Sin `GROUP BY`, la función resume todas las filas filtradas en un solo grupo. Con `GROUP BY`, produce un resultado por cada combinación diferente de columnas agrupadas.

```

1 SELECT department_id,
2        ROUND(AVG(salary), 0) AS promedio_sueldo
3 FROM oehr_employees
4 GROUP BY department_id
5 ORDER BY department_id;
```

Regla esencial: toda expresión del `SELECT` que no sea agregada debe ser compatible con las expresiones del `GROUP BY`.

```

1 SELECT department_id,
2        job_id,
3        TO_CHAR(SUM(salary), 'FM$999,999,990.00') AS total
4 FROM oehr_employees
5 GROUP BY department_id, job_id
6 ORDER BY department_id, job_id;
```

8.3. WHERE frente a HAVING

WHERE filtra filas antes del agrupamiento; HAVING filtra grupos después de calcular las funciones de grupo.

```

1 SELECT department_id,
2     MAX(salary) AS sueldo_maximo
3 FROM oehr_employees
4 WHERE department_id BETWEEN 50 AND 90
5 GROUP BY department_id
6 HAVING MAX(salary) > 10000
7 ORDER BY department_id DESC;
```

Lectura paso a paso:

1. se conservan empleados de departamentos 50 a 90;
2. se forman grupos por departamento;
3. se calcula el máximo de cada grupo;
4. se conservan grupos cuyo máximo supera 10000;
5. se ordena el resultado.

Conexión con la primera prueba

Una consulta de la pauta intenta colocar `MAX(salary)-MIN(salary) <>0` en WHERE. Eso es incorrecto porque la condición sólo existe después de formar el grupo. La forma conceptual correcta es:

```

1 SELECT job_id,
2     COUNT(employee_id) AS cantidad_empleados,
3     ROUND(AVG(salary), 2) AS promedio_sueldo
4 FROM oehr_employees
5 GROUP BY job_id
6 HAVING MAX(salary) - MIN(salary) <> 0;
```

8.4. Lectura activa del módulo 5

M5.1 Muy fácil. ¿Qué función calcula un promedio?

M5.2 Muy fácil. ¿Qué diferencia hay entre `COUNT(*)` y `COUNT(commission_pct)`?

M5.3 Muy fácil. ¿Qué cláusula filtra grupos?

M5.4 Intermedia. Muestra cada `job_id` y la cantidad de empleados que lo poseen.

M5.5 Intermedia. Muestra los departamentos cuyo sueldo promedio es mayor que 8000.

M5.6 Difícil. Para los departamentos 50, 80 y 90, muestra departamento, cargo, cantidad de empleados y promedio de sueldo. Conserva sólo combinaciones con al menos dos empleados y ordena por promedio descendente.

9. Módulo 6: consultas de varias tablas y JOIN

Objetivo de lectura

Combinar tablas sin producir resultados inventados, usando claves, alias y el tipo de unión correcto.

Un JOIN no consiste simplemente en colocar dos tablas juntas: define qué pares de filas representan una misma relación del mundo modelado. Supón que el requerimiento pide código del trabajador, nombre completo y nombre del departamento. Primero eliges `OEHR_EMPLOYEES` como tabla principal porque el sujeto del listado es el trabajador. Allí están el código y los nombres, pero sólo aparece `DEPARTMENT_ID`; el texto `DEPARTMENT_NAME` vive en `OEHR_DEPARTMENTS`. Entonces asignas alias para expresar roles, `e` para empleado y `d` para departamento, y conectas la referencia del empleado con la clave del departamento: `e.department_id=d.department_id`. Oracle toma una fila de empleado, busca la fila departamental cuyo código coincide y forma una fila combinada que contiene columnas de ambas. Después `SELECT` puede escoger `e.employee_id`, los nombres de `e` y `d.department_name`. Si omites la condición, cada empleado se combina con cada departamento; si escribes `department_id=department_id` sin alias, la intención es ambigua; y si conectas `e.manager_id` con `d.manager_id`, estarías respondiendo otra pregunta, no la pertenencia del empleado a un departamento. El tipo de unión depende de qué filas deben sobrevivir. Un `INNER JOIN` conserva sólo empleados con departamento coincidente. Un `LEFT JOIN`, colocando empleados a la izquierda, conserva además al trabajador sin departamento y completa las columnas de `d` con nulos. Antes de elegir, busca expresiones como “todos los empleados”, “aunque no tengan” o “sólo los que poseen”. Finalmente, anticipa la cardinalidad: muchos empleados pueden coincidir con un departamento, de modo que el departamento se repite legítimamente en varias filas. Esa repetición no es un duplicado erróneo; representa distintos empleados relacionados con la misma entidad.

9.1. Por qué aparece un producto cartesiano

Si se escriben varias tablas sin indicar cómo se relacionan, cada fila de una se combina con cada fila de la otra. Ese producto cartesiano suele producir datos incoherentes.

```
1 -- Incorrecto para relacionar empleados y cargos:
2 SELECT employee_id, job_title
3 FROM oehr_employees, oehr_jobs;
```

9.2. INNER JOIN con ON

Un `INNER JOIN` devuelve coincidencias entre ambas tablas.

```
1 SELECT e.employee_id,
2        e.first_name || ' ' || e.last_name AS empleado,
3        j.job_title,
4        d.department_name
5 FROM oehr_employees e
6 JOIN oehr_jobs j
7     ON e.job_id = j.job_id
8 JOIN oehr_departments d
9     ON e.department_id = d.department_id;
```

Los alias `e`, `j` y `d` permiten calificar columnas y eliminan ambigüedades. Nunca escribas `ON(job_id = job_id)`: parece comparar la columna consigo misma y, si el nombre existe en ambas tablas, Oracle informa ambigüedad.

9.3. USING y NATURAL JOIN

USING(columna) es posible cuando la columna de unión tiene el mismo nombre en ambas tablas. Dentro de **USING** no se califica la columna con alias.

```
1 SELECT e.employee_id, d.department_name
2 FROM oehr_employees e
3 JOIN oehr_departments d
4 USING (department_id);
```

NATURAL JOIN une automáticamente todas las columnas de igual nombre. Puede ser breve, pero es menos explícito y puede cambiar de significado si el esquema incorpora otra columna coincidente. Para la prueba, **JOIN ... ON** suele mostrar mejor tu razonamiento.

9.4. LEFT OUTER JOIN

Una unión interna elimina al empleado cuyo **DEPARTMENT_ID** sea nulo o no encuentre coincidencia. Si el requerimiento dice “aunque no tenga departamento”, conserva la tabla de empleados con **LEFT JOIN**.

```
1 SELECT e.employee_id,
2        e.last_name,
3        d.department_name
4 FROM oehr_employees e
5 LEFT JOIN oehr_departments d
6 ON e.department_id = d.department_id;
```

La tabla escrita a la izquierda es la que se preserva. **RIGHT OUTER JOIN** preserva la derecha; **FULL OUTER JOIN** preserva filas no coincidentes de ambas.

9.5. Self join para empleado y jefe

La misma tabla cumple dos roles. Se le asignan dos alias diferentes: **e** es el empleado y **jefe** es la fila del jefe.

```
1 SELECT e.employee_id,
2        e.first_name || ' ' || e.last_name AS empleado,
3        jefe.first_name || ' ' || jefe.last_name AS jefe
4 FROM oehr_employees e
5 LEFT JOIN oehr_employees jefe
6 ON e.manager_id = jefe.employee_id;
```

Se usa **LEFT JOIN** para no perder al empleado superior que no tiene jefe. Si el enunciado sólo pide empleados que sí tienen jefe, un **JOIN** interno es suficiente.

9.6. Cadena de uniones

Para mostrar la región en que trabaja el empleado, sigue cada clave foránea.

```
1 SELECT e.employee_id,
2        e.last_name,
3        r.region_name
4 FROM oehr_employees e
5 JOIN oehr_departments d ON e.department_id = d.department_id
6 JOIN oehr_locations l ON d.location_id = l.location_id
7 JOIN oehr_countries c ON l.country_id = c.country_id
8 JOIN oehr_regions r ON c.region_id = r.region_id;
```

9.7. Taller guiado: construir una unión desde cero

Imagina el requerimiento “mostrar todos los trabajadores, el nombre de su departamento, la ciudad del departamento y el nombre de su cargo; deben aparecer incluso quienes no tengan departamento”. No conviene intentar recordar una consulta completa. Se construye por capas. Primero fija la unidad de salida: una fila por trabajador, de modo que `OEHR_EMPLOYEES` `e` debe iniciar el `FROM`. Segundo, marca dónde vive cada dato: nombre del departamento en `OEHR_DEPARTMENTS` `d`, ciudad en `OEHR_LOCATIONS` `l` y título del cargo en `OEHR_JOBS` `j`. Tercero, dibuja los enlaces: `e.department_id` llega a `d.department_id`; desde esa fila, `d.location_id` llega a `l.location_id`; y `e.job_id` llega directamente a `j.job_id`. Cuarto, decide qué relaciones son opcionales. El enunciado obliga a conservar empleados sin departamento, por lo que el primer enlace departamental es `LEFT JOIN`. La ubicación depende del departamento: si éste falta, también debe poder faltar la ubicación sin eliminar al empleado, por lo que el enlace hacia `l` también es `LEFT JOIN`. En cambio, `JOB_ID` es obligatorio según la definición de empleados; un `JOIN` interno con cargos es razonable. Quinto, escribe el `SELECT` calificando cada columna. Si más tarde se pide sólo empleados de Santiago, existe una decisión semántica importante: `WHERE l.city='Santiago'` eliminará las filas cuya ciudad sea nula y, en la práctica, perderás los empleados sin departamento. Si quieres conservarlos, el requisito debe formularse de otra forma, por ejemplo permitiendo explícitamente `l.city IS NULL`. Esta interacción muestra que elegir `LEFT JOIN` no garantiza por sí sola la preservación: un filtro posterior sobre la tabla opcional puede volver a descartarla. Finalmente, revisa la cantidad esperada de filas. Cada empleado tiene como máximo un departamento, una ubicación y un cargo, así que esta cadena no debería multiplicar a un empleado. Si apareciera repetido, revisa las claves antes de agregar `DISTINCT`.

```

1 SELECT e.employee_id,
2       e.first_name || ' ' || e.last_name AS empleado,
3       d.department_name,
4       l.city,
5       j.job_title
6 FROM oehr_employees e
7 LEFT JOIN oehr_departments d
8   ON e.department_id = d.department_id
9 LEFT JOIN oehr_locations l
10  ON d.location_id = l.location_id
11 JOIN oehr_jobs j
12   ON e.job_id = j.job_id
13 ORDER BY d.department_name, e.last_name;
```

Idea clave

La “X manera” de juntar tablas no se adivina: se determina respondiendo tres preguntas. ¿Cuál es la tabla que define una fila del resultado? ¿Qué pares PK–FK representan el camino hacia los datos que faltan? ¿Deben sobrevivir las filas sin coincidencia? Las respuestas determinan el orden de las tablas, las condiciones `ON` y si corresponde `JOIN` o `LEFT JOIN`.

9.8. Unión no equitativa y `CROSS JOIN`

Una unión no equitativa usa un operador distinto de igualdad, por ejemplo para ubicar un sueldo entre un mínimo y un máximo. Sólo puede practicarse si existe una tabla de rangos adecuada. `CROSS JOIN` expresa deliberadamente un producto cartesiano; no debe aparecer por accidente.

9.9. Lectura activa del módulo 6

M6.1 Muy fácil. ¿Qué cláusula indica la condición que conecta dos tablas?

M6.2 Muy fácil. ¿Para qué sirven los alias de tabla?

M6.3 Muy fácil. ¿Qué tipo de unión conserva empleados sin departamento?

M6.4 Intermedia. Muestra código, apellido y título del cargo de cada empleado.

M6.5 Intermedia. Muestra apellido del empleado, departamento y ciudad.

M6.6 Difícil. Muestra código, nombre completo, nombre del departamento, nombre completo del jefe del departamento, ciudad y título del cargo. Conserva al empleado aunque su departamento o el jefe del departamento sea nulo.

10. Módulo 7: subconsultas

Objetivo de lectura

Usar el resultado de una consulta como criterio de otra consulta y decidir si corresponde =, IN u otro operador.

Una subconsulta resuelve una pregunta auxiliar cuyo resultado necesita la consulta principal. La forma más segura de construirla es verbalizar dos preguntas separadas. Para “mostrar todos los empleados que trabajan en los departamentos de Whalen o King”, la pregunta interior es “¿cuáles son los códigos de departamento de Whalen o King?” y devuelve una lista de `DEPARTMENT_ID`. La pregunta exterior es “¿qué empleados tienen un código contenido en esa lista?”. Como puede haber más de un código, se usa `IN`; emplear `=` supondría una sola fila y produciría un error si aparecen varias. El hecho de que una consulta esté escrita dentro de otra no cambia las reglas ya aprendidas: la interna posee su propio `SELECT`, `FROM` y `WHERE`, y puede incluir agrupamiento. Conviene ejecutarla mentalmente o por separado, comprobar qué columna y cuántas filas produce, y recién entonces elegir el operador exterior. Una subconsulta que calcula `AVG(salary)` sin agrupar garantiza un solo valor y puede compararse con `>`; una que selecciona cargos de un departamento normalmente produce varios valores y pide `IN`. Luego debes distinguir subconsulta de unión. Una unión aporta columnas de otra tabla a cada fila de salida; una subconsulta usada en `WHERE` aporta un criterio para decidir qué filas pasan. Ambas pueden convivir: la consulta exterior une departamentos para mostrar su nombre, mientras la interior descubre los códigos permitidos. Presta atención a los nulos con `NOT IN`: si la lista interior contiene un nulo, las comparaciones pueden quedar desconocidas. En ejercicios introductorios puedes filtrar `IS NOT NULL`; más adelante `NOT EXISTS` ofrece otra forma de expresar ausencia. El objetivo no es anidar por complejidad, sino dividir un problema grande en preguntas pequeñas cuyos resultados encajan por tipo y cardinalidad.

Una subconsulta es una consulta encerrada entre paréntesis. Primero se razona la consulta interior y luego se usa su resultado en la exterior.

10.1. Subconsulta de una fila

Si la consulta interior garantiza un solo valor, puede compararse con `=`, `>`, etc.

```

1 SELECT employee_id, last_name, salary
2 FROM oehr_employees
3 WHERE salary > (
4     SELECT AVG(salary)
5     FROM oehr_employees
6 );
```

La consulta interior produce un promedio; la exterior conserva empleados sobre ese promedio.

10.2. Subconsulta de varias filas con IN

Si la subconsulta puede devolver varios departamentos, `=` no sirve; se necesita `IN`.

```

1 SELECT e.employee_id,
2       e.first_name || ' ' || e.last_name AS empleado,
3       d.department_name
4 FROM oehr_employees e
5 JOIN oehr_departments d
6     ON e.department_id = d.department_id
7 WHERE e.department_id IN (
8     SELECT department_id
9     FROM oehr_employees
10    WHERE UPPER(last_name) IN ('WHALEN', 'KING')
11 );

```

Éste es el patrón de la quinta consulta visible en la pauta. La subconsulta responde “¿en qué departamentos trabajan Whalen o King?”; la consulta externa obtiene todos los empleados de esos departamentos.

10.3. Subconsultas y valores nulos

NOT IN puede dar resultados inesperados si la subconsulta devuelve algún nulo, porque la comparación pasa a ser desconocida. Cuando exista esa posibilidad, filtra los nulos en la subconsulta o estudia NOT EXISTS. Para el patrón confirmado de la pauta se usa IN, por lo que el problema no es central, pero conviene reconocerlo.

10.4. Lectura activa del módulo 7

M7.1 Muy fácil. ¿Entre qué signos se escribe una subconsulta?

M7.2 Muy fácil. Si la subconsulta devuelve varios valores, ¿qué operador permite comparar contra todos ellos?

M7.3 Muy fácil. ¿Qué consulta debes comprender primero: la interna o la externa?

M7.4 Intermedia. Muestra empleados que ganan más que el sueldo promedio del departamento 90.

M7.5 Intermedia. Muestra empleados cuyo cargo sea uno de los cargos que existen en el departamento 50.

M7.6 Difícil. Muestra código, nombre y departamento de quienes trabajan en un departamento que contiene al menos un empleado cuyo sueldo supera 12000. No escribas manualmente los códigos de departamento.

11. Pauta razonada de los seis requerimientos

Esta pauta se alinea con los seis requerimientos del documento de evaluación. No debe memorizarse literalmente: primero resuelve cada ejercicio, luego compara la estructura y explica con tus propias palabras por qué se utilizó cada cláusula.

11.1. Ejercicio 1: texto y patrón

Habilidades: concatenación, UPPER, LIKE, OR.

```

1 SELECT 'El código del cargo ' || job_title ||
2       ' es: ' || job_id AS "CARGO",
3       min_salary AS "SUELDO MINIMO"
4 FROM oehr_jobs
5 WHERE UPPER(job_title) LIKE 'S%'
6       OR UPPER(job_id) LIKE 'S%';

```

Comprueba que la frase, el sueldo mínimo y ambos criterios de búsqueda estén presentes. El nombre definido en el esquema es OEHR_JOBS.

11.2. Ejercicio 2: resumen por cargo

Habilidades: COUNT, AVG, MIN, MAX, GROUP BY y HAVING.

```
1 SELECT job_id AS "OFICIO",
2        COUNT(employee_id) AS "CANTIDAD EMPLEADOS",
3        ROUND(AVG(salary), 2) AS "PROMEDIO DE SUELDOS",
4        MAX(salary) - MIN(salary) AS "DIFERENCIA SUELDOS"
5 FROM oehr_employees
6 GROUP BY job_id
7 HAVING MAX(salary) - MIN(salary) <> 0;
```

La condición agregada pertenece a HAVING, porque se evalúa después de formar cada grupo. La diferencia también debe aparecer en la salida, ya que el enunciado la solicita explícitamente.

11.3. Ejercicio 3: empleado, departamento y jefe

Habilidades: unión normal, auto-unión y alias de tabla. El enunciado pide al jefe del departamento, por lo que el segundo enlace nace en d.manager_id; no en e.manager_id, que representa al jefe directo del empleado.

```
1 SELECT e.employee_id,
2        e.first_name || ' ' || e.last_name AS empleado,
3        d.department_name,
4        jefe.first_name || ' ' || jefe.last_name AS jefe
5 FROM oehr_employees e
6 JOIN oehr_departments d
7     ON e.department_id = d.department_id
8 JOIN oehr_employees jefe
9     ON d.manager_id = jefe.employee_id;
```

Si una variante exigiera conservar empleados sin departamento o departamentos sin jefe, se usarían LEFT JOIN. En la respuesta de referencia se emplea JOIN, porque se solicitan datos asociados de las tres filas.

11.4. Ejercicio 4: promedio del mínimo salarial por cargo

El fragmento de pauta es ambiguo: OEHR_JOBS posee una sola fila por JOB_ID, de modo que promediar MIN_SALARY por ese mismo código no agrega información. Una reconstrucción sintácticamente válida es:

```
1 SELECT job_id,
2        AVG(min_salary) AS promedio_minimo
3 FROM oehr_jobs
4 GROUP BY job_id
5 HAVING AVG(min_salary) > 3000;
```

Pedagógicamente, lo importante es reconocer que una condición sobre AVG se escribe en HAVING. En una prueba real se debe leer el enunciado completo antes de decidir si la tabla o el agrupamiento son realmente esos.

11.5. Ejercicio 5: departamentos de Whalen o King

Habilidades: JOIN, concatenación y subconsulta multirregistro con IN.

```

1 SELECT e.employee_id,
2       e.first_name || ' ' || e.last_name AS empleado,
3       d.department_name
4 FROM oehr_employees e
5 JOIN oehr_departments d
6     ON e.department_id = d.department_id
7 WHERE e.department_id IN (
8     SELECT department_id
9     FROM oehr_employees
10    WHERE UPPER(last_name) IN ('WHALEN', 'KING')
11 )
12 ORDER BY d.department_name, e.last_name;

```

La subconsulta puede devolver más de un departamento: King no identifica necesariamente una única fila. Por esa razón se compara con IN y no con el operador =.

11.6. Ejercicio 6: comisión, rango, correo y departamento de Kochhar

Habilidades: concatenación, SUBSTR, nulos, NOT BETWEEN, conectores lógicos y subconsulta.

```

1 SELECT e.first_name || ' ' || e.last_name AS "NOMBRE EMPLEADO",
2       SUBSTR(e.first_name, 1, 1) ||
3       SUBSTR(e.last_name, 2, 1) ||
4       e.job_id || '@pruebasql.cl' AS "NUEVO CORREO",
5       e.commission_pct AS "COMISION"
6 FROM oehr_employees e
7 WHERE e.commission_pct IS NOT NULL
8       AND e.salary NOT BETWEEN 5000 AND 10000
9       AND e.department_id IN (
10        SELECT department_id
11        FROM oehr_employees
12        WHERE UPPER(first_name) = 'NEENA'
13              AND UPPER(last_name) = 'KOCHHAR'
14 );

```

La frase “tengan una comisión asociada” se interpreta como `commission_pct IS NOT NULL`. “Fuera del rango” se expresa con `NOT BETWEEN`; los extremos 5000 y 10000 quedan fuera del resultado porque `BETWEEN` es inclusivo.

12. Lista de errores que debes poder diagnosticar

Error	Corrección razonada
OEHR_JOBS	El esquema define OEHR_JOBS.
DEPARMENT_ID	La columna es DEPARTMENT_ID.
Alias con comillas simples	Usa alias sin comillas o con comillas dobles.
Falta una coma en SELECT	Cada expresión seleccionada se separa con coma.
Punto y coma antes de terminar	Escribe un único punto y coma al final.
WHERE MAX(salary)>...	Una condición agregada corresponde a HAVING.
Columna normal junto a AVG sin agrupar	Agrega la columna a GROUP BY o quítala del SELECT.
commission_pct = NULL	Usa IS NULL.
ON(job_id=job_id)	Califica ambos lados con alias de tabla.
Varias tablas sin unión	Escribe cada relación mediante JOIN ... ON.
CONCAT(a,b,c)	Usa a b c o anida CONCAT.
Fecha como texto ambiguo	Usa TO_DATE(texto,formato).

Error	Corrección razonada
MOD(54/3)	La función recibe dos argumentos: MOD(54,3).
Cadena de <> unida con OR	Usa NOT IN (...) o une correctamente con AND.

13. Pauta de autoevaluación alineada con la rúbrica

Para cada fila marca los criterios comprobados y luego asigna un nivel: Destacado = 5, Competente = 4, Básico = 3 y En desarrollo = 1. Los criterios internos orientan la corrección, pero cada fila aporta como máximo 5 puntos. La suma máxima es 35.

Ítem	Criterios observables	Nivel
Formalidad	☐ archivo .sql; ☐ nombre del archivo correcto; ☐ encabezado con datos; ☐ un comentario por requerimiento; ☐ código ordenado y entregable.	/5
R1	☐ muestra frase y sueldo mínimo; ☐ concatena; ☐ usa UPPER; ☐ filtra con LIKE 'S%'; ☐ conecta ambos criterios con OR.	/5
R2	☐ muestra las cuatro métricas; ☐ usa COUNT y AVG; ☐ calcula MAX-MIN; ☐ agrupa por oficio; ☐ filtra la diferencia con HAVING.	/5
R3	☐ muestra los cuatro datos; ☐ usa dos JOIN; ☐ escribe cada ON; ☐ usa alias de tabla; ☐ obtiene al jefe desde d.manager_id.	/5
R4	☐ muestra el código de oficio; ☐ calcula AVG(min_salary); ☐ agrupa por oficio; ☐ usa HAVING; ☐ compara el promedio con 3000.	/5
R5	☐ muestra código, nombre y departamento; ☐ concatena nombre y apellido; ☐ une empleados y departamentos; ☐ la subconsulta busca Whalen o King; ☐ compara con IN.	/5
R6	☐ construye el correo pedido; ☐ exige comisión con IS NOT NULL; ☐ usa NOT BETWEEN; ☐ combina condiciones con AND; ☐ obtiene el departamento de Neena Kochhar mediante subconsulta.	/5
Total	Escribe el error que debes corregir primero en el próximo intento:	/35

Pausa activa

No te asignes puntaje sólo porque la consulta “se parece” a la pauta. Comprueba que ejecuta, que las columnas corresponden al enunciado y que las filas obtenidas cumplen todas las condiciones. Si no puedes ejecutarla, recorre en papel las seis decisiones y revisa cada relación de claves.

14. Simulacro acumulativo

Resuelve sin mirar el solucionario. Antes de escribir SQL, anota las seis decisiones del método inicial.

Parte A: muy fácil

S.1 Muestra código, apellido, sueldo y departamento de todos los empleados.

S.2 Muestra los códigos de cargo distintos presentes en empleados.

S.3 Muestra los empleados que no reciben comisión.

Parte B: intermedia

- S.4** Muestra código, nombre completo en mayúsculas, correo terminado en @BD.UCM.CL y fecha de contratación con formato DD-MM-YYYY, para departamentos 50 y 80. Ordena desde la contratación más reciente.
- S.5** Muestra nombre de departamento, cantidad de empleados, sueldo mínimo, máximo y promedio. Conserva departamentos cuyo promedio sea mayor que 7000.

Parte C: difícil

- S.6** Muestra el código y nombre del empleado, título de su cargo, nombre de su departamento, ciudad, región y nombre de su jefe directo. Conserva empleados sin jefe o sin departamento. La salida debe estar ordenada por región, departamento y apellido.

15. Solucionario razonado

No uses este apartado como lectura pasiva. Para cada respuesta, señala con un lápiz qué parte corresponde a salida, origen, relación, filtro de fila, agrupamiento, filtro de grupo y orden.

15.1. Soluciones del módulo 1

M1.1 SELECT.

M1.2 El asterisco *.

M1.3 DISTINCT.

M1.4

```

1 SELECT employee_id AS "CODIGO",
2       last_name AS "APELLIDO",
3       salary * 12 AS "ANUAL"
4 FROM oehr_employees;
```

M1.5 La primera suma sólo 1200 al sueldo por la precedencia de la multiplicación. La segunda suma 100 al sueldo mensual y luego anualiza todo.

M1.6

```

1 SELECT UPPER(last_name) || ' gana ' ||
2       TO_CHAR(salary * 12, 'FM999999990') ||
3       ' al anio' AS reporte
4 FROM oehr_employees;
```

15.2. Soluciones del módulo 2

M2.1 IS NULL.

M2.2 Exactamente un carácter.

M2.3 DESC.

M2.4

```

1 SELECT employee_id, last_name, department_id, salary
2 FROM oehr_employees
3 WHERE department_id IN (50, 80)
4       AND salary BETWEEN 4000 AND 9000;
```

M2.5 WHERE LOWER(last_name) LIKE '_a%s'.

M2.6

```

1 SELECT employee_id, last_name, department_id, salary
2 FROM oehr_employees
3 WHERE department_id IN (50, 80, 90)
4       AND commission_pct IS NULL
5 ORDER BY department_id ASC, salary DESC;
```

15.3. Soluciones del módulo 3

M3.1 UPPER.

M3.2 ROUND redondea; TRUNC elimina las posiciones sobrantes sin redondear.

M3.3 Para evaluar una expresión una vez cuando no depende de una tabla de datos del usuario.

M3.4

```

1 SELECT employee_id,
2     LOWER(SUBSTR(first_name, 1, 2) || '.' ||
3         last_name || '@ucm.cl') AS correo
4 FROM oehr_employees;

```

```

1 SELECT employee_id,
2     TRUNC(MONTHS_BETWEEN(SYSDATE, hire_date) / 12) AS anios
3 FROM oehr_employees;

```

M3.6

```

1 SELECT employee_id,
2     UPPER(SUBSTR(last_name, -1, 1) ||
3         SUBSTR(first_name, 1, 1)) ||
4     TO_CHAR(LENGTH(first_name || last_name)) AS identificador
5 FROM oehr_employees;

```

15.4. Soluciones del módulo 4**M4.1** TO_CHAR.**M4.2** Cero.**M4.3** COALESCE.**M4.4**

```

1 SELECT employee_id,
2     TO_CHAR(salary, 'FM999999990.00') AS sueldo,
3     TO_CHAR(hire_date, 'YYYY-MM-DD') AS fecha
4 FROM oehr_employees;

```

```

1 SELECT employee_id,
2     CASE
3         WHEN commission_pct IS NULL THEN 'SIN COMISION'
4         ELSE 'CON COMISION'
5     END AS estado
6 FROM oehr_employees;

```

M4.6

```

1 SELECT first_name || ' ' || last_name AS empleado,
2     TO_CHAR(salary, 'FM$999,999,990.00') AS sueldo,
3     CASE
4         WHEN commission_pct IS NULL THEN 'SIN COMISION'
5         ELSE TO_CHAR(salary * commission_pct,
6             'FM$999,999,990.00')
7     END AS monto_comision
8 FROM oehr_employees;

```

15.5. Soluciones del módulo 5**M5.1** AVG.**M5.2** COUNT(*) cuenta filas; COUNT(commission_pct) sólo cuenta filas donde esa columna no es nula.**M5.3** HAVING.

M5.4

```

1 SELECT job_id, COUNT(*) AS cantidad
2 FROM oehr_employees
3 GROUP BY job_id;

```

```

1 SELECT department_id, ROUND(AVG(salary), 2) AS promedio
2 FROM oehr_employees
3 GROUP BY department_id
4 HAVING AVG(salary) > 8000;

```

M5.6

```

1 SELECT department_id,
2        job_id,
3        COUNT(*) AS cantidad,
4        ROUND(AVG(salary), 2) AS promedio
5 FROM oehr_employees
6 WHERE department_id IN (50, 80, 90)
7 GROUP BY department_id, job_id
8 HAVING COUNT(*) >= 2
9 ORDER BY promedio DESC;

```

15.6. Soluciones del módulo 6

M6.1 ON; también puede utilizarse USING cuando corresponda.

M6.2 Para representar el rol de cada tabla, acortar sus nombres y evitar columnas ambiguas.

M6.3 LEFT JOIN desde empleados hacia departamentos.

M6.4

```

1 SELECT e.employee_id, e.last_name, j.job_title
2 FROM oehr_employees e
3 JOIN oehr_jobs j ON e.job_id = j.job_id;

```

```

1 SELECT e.last_name, d.department_name, l.city
2 FROM oehr_employees e
3 JOIN oehr_departments d ON e.department_id = d.department_id
4 JOIN oehr_locations l ON d.location_id = l.location_id;

```

M6.6 El “jefe del departamento” se obtiene desde el identificador de jefe de la tabla departamentos (alias d), no necesariamente desde el jefe directo del empleado.

```

1 SELECT e.employee_id,
2        e.first_name || ' ' || e.last_name AS empleado,
3        d.department_name,
4        jd.first_name || ' ' || jd.last_name AS jefe_departamento,
5        l.city,
6        c.job_title
7 FROM oehr_employees e
8 JOIN oehr_jobs c
9     ON e.job_id = c.job_id
10 LEFT JOIN oehr_departments d
11     ON e.department_id = d.department_id
12 LEFT JOIN oehr_employees jd
13     ON d.manager_id = jd.employee_id
14 LEFT JOIN oehr_locations l
15     ON d.location_id = l.location_id;

```

15.7. Soluciones del módulo 7

M7.1 Entre paréntesis.

M7.2 IN.

M7.3 La interna, porque su resultado alimenta la condición externa.

M7.4

```
1 SELECT employee_id, last_name, salary
2 FROM oehr_employees
3 WHERE salary > (
4     SELECT AVG(salary)
5     FROM oehr_employees
6     WHERE department_id = 90
7 );
```

```
1 SELECT employee_id, last_name, job_id
2 FROM oehr_employees
3 WHERE job_id IN (
4     SELECT job_id
5     FROM oehr_employees
6     WHERE department_id = 50
7 );
```

M7.6

```
1 SELECT e.employee_id,
2        e.first_name || ' ' || e.last_name AS empleado,
3        d.department_name
4 FROM oehr_employees e
5 JOIN oehr_departments d
6     ON e.department_id = d.department_id
7 WHERE e.department_id IN (
8     SELECT department_id
9     FROM oehr_employees
10    WHERE salary > 12000
11        AND department_id IS NOT NULL
12 );
```

15.8. Soluciones del simulacro

S.1

```
1 SELECT employee_id, last_name, salary, department_id
2 FROM oehr_employees;
```

```
1 SELECT DISTINCT job_id
2 FROM oehr_employees;
```

S.3

```
1 SELECT employee_id, last_name
2 FROM oehr_employees
3 WHERE commission_pct IS NULL;
```

```
1 SELECT employee_id,
2        UPPER(first_name || ' ' || last_name) AS nombre,
3        email || '@BD.UCM.CL' AS correo,
4        TO_CHAR(hire_date, 'DD-MM-YYYY') AS fecha_contrato
5 FROM oehr_employees
6 WHERE department_id IN (50, 80)
7 ORDER BY hire_date DESC;
```

S.4 Se usa una unión para mostrar el nombre y se agrupa por identificador y nombre para no mezclar departamentos distintos que, hipotéticamente, tuvieran el mismo nombre.

```

1 SELECT d.department_id,
2        d.department_name,
3        COUNT(e.employee_id) AS cantidad,
4        MIN(e.salary) AS minimo,
5        MAX(e.salary) AS maximo,
6        ROUND(AVG(e.salary), 2) AS promedio
7 FROM oehr_departments d
8 JOIN oehr_employees e
9     ON d.department_id = e.department_id
10 GROUP BY d.department_id, d.department_name
11 HAVING AVG(e.salary) > 7000;

```

S.6 La preservación de empleados exige que todas las tablas opcionales cuelguen mediante LEFT JOIN desde la cadena preservada.

```

1 SELECT e.employee_id,
2        e.first_name || ' ' || e.last_name AS empleado,
3        j.job_title,
4        d.department_name,
5        l.city,
6        r.region_name,
7        jefe.first_name || ' ' || jefe.last_name AS jefe
8 FROM oehr_employees e
9 JOIN oehr_jobs j
10     ON e.job_id = j.job_id
11 LEFT JOIN oehr_departments d
12     ON e.department_id = d.department_id
13 LEFT JOIN oehr_locations l
14     ON d.location_id = l.location_id
15 LEFT JOIN oehr_countries c
16     ON l.country_id = c.country_id
17 LEFT JOIN oehr_regions r
18     ON c.region_id = r.region_id
19 LEFT JOIN oehr_employees jefe
20     ON e.manager_id = jefe.employee_id
21 ORDER BY r.region_name, d.department_name, e.last_name;

```

16. Hoja de repaso final

Antes de entregar una consulta, verifica:

1. ¿Escribí exactamente OEHR_EMPLOYEES, OEHR_DEPARTMENTS y OEHR_JOBS?
2. ¿Cada columna pertenece realmente a la tabla cuyo alias utilicé?
3. ¿Separé todas las expresiones del SELECT con comas?
4. ¿Las cadenas tienen comillas simples y los alias especiales comillas dobles?
5. ¿Comparé los nulos con IS NULL o IS NOT NULL?
6. ¿Normalicé mayúsculas y minúsculas en búsquedas textuales?
7. ¿Usé WHERE para filas y HAVING para resultados agregados?
8. ¿Incluí en GROUP BY todas las columnas no agregadas del resultado?

9. ¿Cada JOIN tiene una relación de clave correcta y calificada con alias?
10. ¿Elegí LEFT JOIN cuando debían conservarse filas sin coincidencia?
11. ¿Usé IN si una subconsulta puede devolver varias filas?
12. ¿Dejé un único punto y coma al final?

Idea clave

La mejor forma de estudiar no es memorizar el orden visual de un ejemplo. Toma un requerimiento, subraya sustantivos para descubrir columnas y tablas, encierra las restricciones de filas, marca las palabras “por cada” para descubrir agrupamientos y busca “promedio”, “cantidad”, “máximo”, “mínimo” o “total” para descubrir funciones de grupo.

17. Documentación oficial utilizada

Las siguientes páginas pertenecen a *Oracle Database 21c SQL Language Reference*. Se usaron para comprobar la sintaxis y complementar los scripts del curso:

- [SELECT y sus cláusulas](#);
- [condiciones SQL](#);
- [condiciones de patrones LIKE](#);
- [condiciones de valores nulos](#);
- [funciones de una sola fila](#);
- [funciones de agregación](#);
- [uniones](#);
- [expresiones de subconsulta escalar](#);
- [TO_CHAR para fechas](#);
- [TO_DATE](#);
- [NVL y NVL2](#);
- [expresiones CASE y DECODE](#);
- [funciones de fecha y hora](#).

Nota de alcance. La documentación oficial describe muchas capacidades que exceden esta evaluación. La selección de temas de esta guía se restringe a lo enseñado en U1 L1–L6 y a las habilidades visibles en la pauta. No se incluyen procedimientos PL/SQL, triggers, control de concurrencia ni administración, porque no aparecen en los materiales de esta primera prueba.